Stage Neural Compression Pipeline

Case 2- Luddy Hackathon 2026

Team Name: CodeNova
Team members:
Aishwarya Balasaheb Dhaigude, Koushik Ballamudi, Madhumita Gannavaram, Nileet Savale

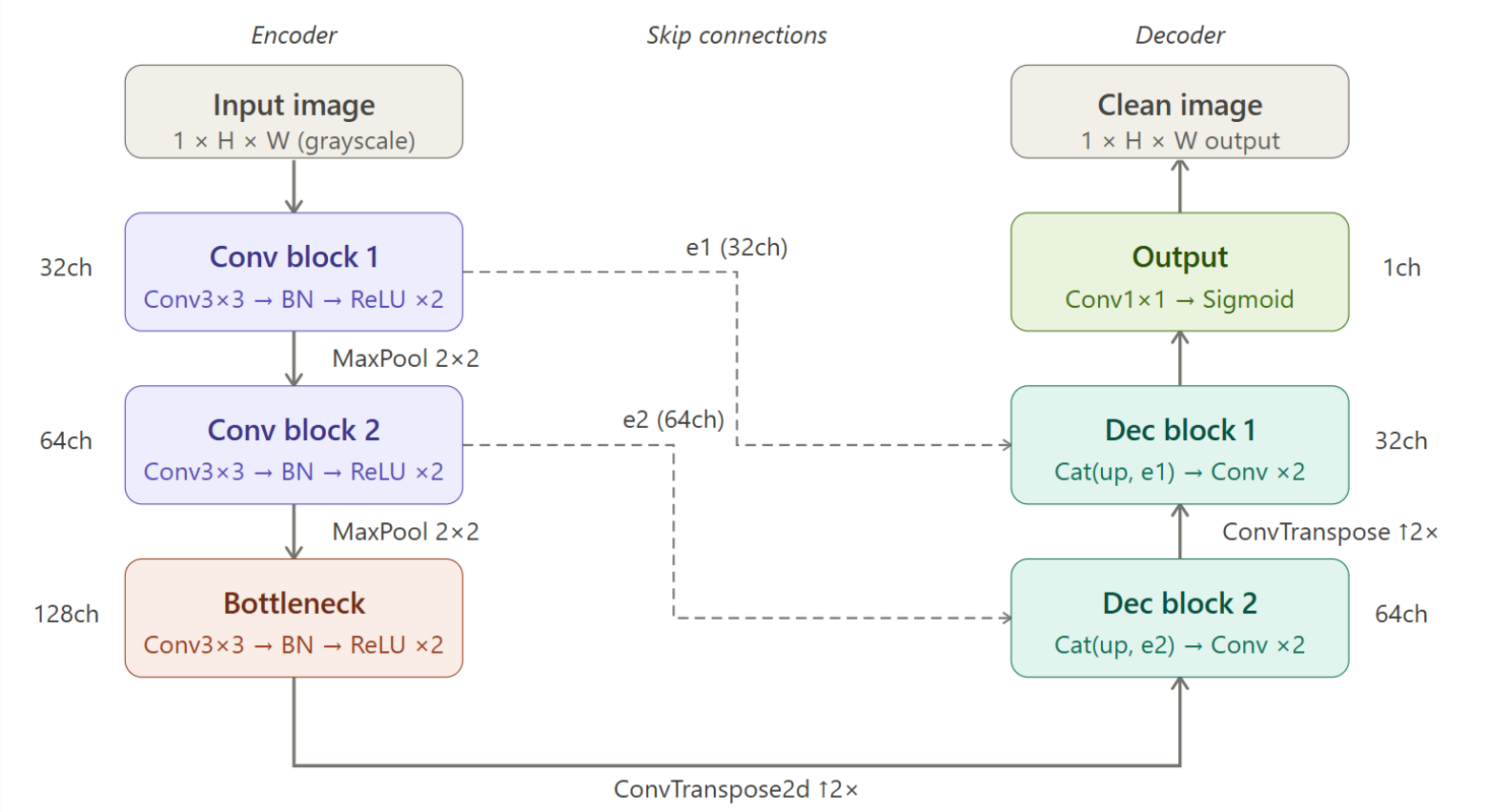
DenoisingCNN

Architecture Documentation

U-Net Style Convolutional Neural Network · Image Denoising for OCR Pre-processing

Stage 1 · CNN / OCR Microservice

Architecture Overview — U-Net Style Encoder-Decoder



Design Choice 1 — Layer Depth & Feature Channels

01

3-Level Encoder Hierarchy

The encoder progressively doubles channels: 32 → 64 → 128. This allows the network to learn low-level edge/texture features in early layers and high-level structural features (distortion patterns) in deeper layers — essential for document denoising where noise manifests at multiple scales.

02

2 Conv Layers per Block

Each encoder/decoder block contains two consecutive Conv+BN+ReLU operations before pooling. Stacking two 3×3 convolutions gives a 5×5 effective receptive field while using fewer parameters than a single 5×5 conv — a proven design pattern from VGGNet applied here to extract richer features per resolution level.

03

128-Channel Bottleneck

The deepest layer uses 128 channels at the lowest spatial resolution (64×64 for a 256×256 input). This compressed representation forces the network to encode the most discriminative content-vs-noise features. A wider bottleneck risks overfitting; narrower risks underfitting on the four noise types (crumple, fold, wrinkle, pepper).

Design Choice 2 — Kernel Sizes & Spatial Padding

Kernel Size	1×1	3×3 ✓	5×5	7×7
Receptive Field	Pixel-wise	**Local 3×3**	Local 5×5	Large 7×7
Parameters	Minimal	**Moderate**	High	Very High
For Denoising	Too narrow	**Optimal**	Redundant	Overkill

padding=1 on all Conv2d

Same-padding preserves spatial dimensions through convolution (H×W → H×W). Without it, each 3×3 conv would shrink the feature map by 2px per side, creating mismatches during skip-connection concatenation and distorting spatial alignment of fine text strokes.

1×1 Conv for Output Layer

The final Conv2d(32→1, kernel=1) is a channel-projection layer — it collapses 32 feature maps to 1 output channel (grayscale). A 1×1 kernel is optimal here since spatial mixing is already complete; this acts as a per-pixel learned weighting of the decoded features.

Design Choice 3 — Activation Functions & Normalisation

ReLU — Hidden Layers

→ Non-saturating: gradient flows through positive activations without vanishing

→ Sparsity: zero-activation pruning reduces inter-feature noise in early layers

→ Computationally cheap: max(0,x) vs sigmoid/tanh

→ Standard choice for CNNs processing grayscale image patches

Sigmoid — Output Layer

→ Squashes output to [0,1] — matching the [0,1] normalised pixel range

→ Smooth gradient at output ensures MSE loss gradients are stable

→ Prevents pixel blow-up (values > 1.0) that would corrupt PSNR metric

→ Deterministic clipping-free output, unlike ReLU which could saturate to 0

BatchNorm2d — Every Conv

→ Normalises activations per mini-batch: accelerates convergence significantly

→ Acts as regulariser: reduces need for dropout in this architecture

→ Reduces internal covariate shift between encoder and decoder paths

→ Crucial when training on 256×256 patches with high input variance (4 noise types)

Design Choice 5 — Training Configuration

Optimizer

Adam (lr = 0.001)

Adam's adaptive learning rates per parameter suit the mixed-scale features in U-Net. More robust than SGD for tasks where encoder and decoder require different update magnitudes.

Epochs

15

Sufficient for convergence on document denoising; best weights auto-saved per epoch via ReduceLROnPlateau monitoring.

LR Scheduler

ReduceLROnPlateau

Halves LR after 5 epochs of no improvement (factor=0.5). Prevents overshooting in later epochs when the model is fine-tuning texture details.

Patch Size

256 × 256 px

Random crops multiply dataset size by 30× per image. Patches expose the network to diverse local noise patterns while fitting in CUDA memory efficiently.

Loss Function

MSELoss

MSE directly penalises pixel-level reconstruction error, which correlates with PSNR — the evaluation metric for denoising quality. Suitable for regression on [0,1] pixel values.

Patches/Image

30

Balances dataset diversity (30 random crops per image pair) with training speed. Each crop provides an independent noise-to-clean sample.

Design Justification Summary

3×3 Kernels + padding=1	Optimal local receptive field for text strokes; same-padding preserves spatial dims for skip connections
2 Conv layers per block	VGG-style double conv gives 5×5 effective field with fewer params than a single 5×5 conv
32 → 64 → 128 channels	Progressive channel doubling captures features at increasing semantic complexity
MaxPool + ConvTranspose2d	Symmetric down/upsampling ensures pixel-precise reconstruction after skip-connection alignment
BatchNorm2d after every Conv	Stabilises training across 4 noise types; acts as implicit regulariser without dropout
ReLU in hidden layers	Prevents vanishing gradients, introduces sparsity; computationally cheap
Sigmoid at output	Enforces [0,1] output range matching normalised pixel values; compatible with MSE loss
Skip connections (U-Net)	Preserves spatial detail from encoder; prevents blurring of fine character strokes in decoded output

Adaptive Huffman Encoding

Compression Microservice

Adaptive Huffman • FGK • Flask • Port 8002

Stage 1
OCR :8001
sends { text }

POST /compress
:8002
huffman.py + metrics.py

POST /decompress
:8002
base64 → original text

```
GET /health → { status:
'ok', port: 8002 }
```

- **Compression ratio: 1.0 - 3.0+**
- **Entropy: 2.3 - 6.5 bits/symbol**
- **Efficiency: 55% - 87%**
- **Space saving: ~2% - 66%**